

Complete TODO: Iskra + Zvezda

Project Overview

ISKRA

1.2 Python Library Interface (CRITICAL)

Priority: P0 | Effort: 6-8 hours | Blockers: None

Make Iskra importable as a clean Python library for Zvezda.

```
# Zvezda will use this API
from iskra import IskraManager, RepoStatus

manager = IskraManager(config_path=~/.config/iskra")

# Query operations
repos = manager.get_all_repos(active_only=True)
status = manager.get_repo_status("/path/to/repo")
filters = manager.filter_repos(has_changes=True, behind_remote=True)

# Action operations
result = manager.process_repo(
    repo_path="/path/to/repo",
    pull=True,
    commit=True,
    push=True,
    dry_run=False
)

# Batch operations
results = manager.process_all(
    pull=True,
    filters={"has_changes": True}
)
```

Tasks:

- ☐ Create src/auto_commit/api/ module
- ☐ Create IskraManager class:

```
class IskraManager:
    def __init__(self, config_path: Optional[str] = None)

    # Query methods
    def get_all_repos(self, active_only: bool = True) -> List[RepoInfo]
    def get_repo_status(self, repo_path: str) -> RepoStatus
```

```

def filter_repos(self, **filters) -> List[RepoInfo]

# Action methods
def process_repo(self, repo_path: str, **options) -> ProcessResult
def process_all(self, **options) -> BatchResult
def pull_repo(self, repo_path: str) -> PullResult
def commit_repo(self, repo_path: str, message: str) -> CommitResult
def push_repo(self, repo_path: str) -> PushResult

# Config methods
def add_repo(self, repo_path: str) -> bool
def remove_repo(self, repo_path: str) -> bool
def update_repo_config(self, repo_path: str, **config) -> bool

# Validation methods
def validate_repo(self, repo_path: str) -> ValidationResult
def check_large_files(self, repo_path: str) -> List[str]
def scan_secrets(self, repo_path: str) -> List[SecretMatch]

```

- Create data classes:

```

@dataclass
class RepoStatus:
    path: str
    name: str
    branch: str
    changes: ChangesSummary
    remote: RemoteStatus
    last_commit: CommitInfo
    worktrees: List[WorktreeInfo]

@dataclass
class ProcessResult:
    success: bool
    repo_path: str
    operations: List[Operation] # pull, commit, push
    errors: List[str]
    duration: float

```

- Separate UI from logic:
 - Move all git operations to core/git_operations.py
 - Move all formatting to ui/display.py
 - Keep logic in api/manager.py
- Add comprehensive docstrings
- Write unit tests

- ☐ Create `examples/` directory with usage examples
- ☐ Update README with API documentation

Files to create:

- `src/auto_commit/api/__init__.py`
 - `src/auto_commit/api/manager.py`
 - `src/auto_commit/api/types.py` (data classes)
 - `src/auto_commit/api/exceptions.py`
-

1.3 Event Hooks System (HIGH PRIORITY)

Priority: P1 | Effort: 4-5 hours | Blockers: 1.2

Allow Zvezda to register callbacks for real-time updates.

```
from iskra import IskraManager

manager = IskraManager()

# Register event handlers
@manager.on('before_pull')
def on_before_pull(repo_path: str):
    print(f"About to pull {repo_path}")

@manager.on('after_commit')
def on_after_commit(repo_path: str, commit_hash: str, message: str):
    print(f"Committed: {message}")

@manager.on('error')
def on_error(repo_path: str, operation: str, error: Exception):
    print(f"Error in {repo_path}: {error}")

# Run operations (callbacks fire automatically)
manager.process_all()
```

Tasks:

- ☐ Create event system in `src/auto_commit/api/events.py`

```
class EventEmitter:
    def __init__(self)
    def on(self, event: str, handler: Callable) -> None
    def emit(self, event: str, **kwargs) -> None
    def remove_listener(self, event: str, handler: Callable) -> None
```
- ☐ Define standard events:

- before_scan - Before scanning for repos
- after_scan - After scanning completes
- before_pull - Before pulling a repo
- after_pull - After pull completes
- before_commit - Before committing changes
- after_commit - After commit completes
- before_push - Before pushing
- after_push - After push completes
- repo_start - Starting to process a repo
- repo_complete - Finished processing a repo
- repo_skip - Skipping a repo
- error - When an error occurs
- progress - Progress updates (for batch operations)

- ☐ Integrate with IskraManager
- ☐ Add async event support (optional)
- ☐ Document event system
- ☐ Create example listener implementations

Files to create:

- src/auto_commit/api/events.py
- examples/event_listener.py

1.4 Status Query Command (HIGH PRIORITY)

Priority: P1 | Effort: 3-4 hours | Blockers: 1.1

Detailed status without operations - Zvezda needs this constantly.

```
iskra status                # All repos
iskra status --repo /path   # Single repo
iskra status --json         # For Zvezda
iskra status --format table # Rich table (default)
```

Tasks:

- ☐ Create `iskra status` subcommand
- ☐ Implement detailed status gathering:

```
def get_detailed_status(repo_path: str) -> RepoStatus:
    return RepoStatus(
        path=repo_path,
        name=os.path.basename(repo_path),
        branch=get_current_branch(),
        changes=count_changes(),
```

```

        remote=check_remote_status(),
        last_commit=get_last_commit_info(),
        worktrees=list_worktrees(),
        stashes=list_stashes(),
        remotes=list_remotes(),
        is_clean=check_clean(),
        has_conflicts=check_conflicts()
    )

```

☐ Display formats:

- Table view (Rich)
- JSON output
- Compact one-line per repo

☐ Add filters to status:

```

iskra status --has-changes
iskra status --behind-remote
iskra status --on-branch main

```

☐ Show summary statistics

☐ Cache status results (with TTL)

☐ Add --refresh to bypass cache

Files to modify:

- src/auto_commit/auto_commit.py (add status subcommand)
- src/auto_commit/core/git_operations.py (add status functions)

1.5 Smart Filters (HIGH PRIORITY)

Priority: P1 | Effort: 3-4 hours | Blockers: 1.4

State-based repository filtering (not just glob patterns).

```

# State filters
iskra --has-changes           # Has uncommitted changes
iskra --behind-remote         # Behind remote (needs pull)
iskra --ahead-remote          # Ahead remote (needs push)
iskra --on-branch PATTERN     # On specific branch
iskra --dirty                 # Has untracked/modified files
iskra --clean                 # No changes at all
iskra --conflicts             # Has merge conflicts

# Combine filters
iskra --has-changes --on-branch main
iskra --behind-remote --dirty

```

Tasks:

- ☐ Create `src/auto_commit/core/filters.py`:

```
class RepoFilter:
    @staticmethod
    def has_changes(repo_path: str) -> bool

    @staticmethod
    def behind_remote(repo_path: str) -> bool

    @staticmethod
    def ahead_remote(repo_path: str) -> bool

    @staticmethod
    def on_branch(repo_path: str, pattern: str) -> bool

    @staticmethod
    def is_dirty(repo_path: str) -> bool

    @staticmethod
    def is_clean(repo_path: str) -> bool

    @staticmethod
    def has_conflicts(repo_path: str) -> bool

    @classmethod
    def apply_filters(cls, repos: List[str], **filters) -> List[str]
```

- ☐ Add filter arguments to CLI
- ☐ Integrate with `IskraManager.filter_repos()`
- ☐ Add filter result preview:
Filters applied: `--has-changes --on-branch main`
Matched 12 of 50 repositories
- ☐ Support negation: `--not-clean, --not-on-branch`
- ☐ Add to JSON output
- ☐ Document all filters

Files to create:

- `src/auto_commit/core/filters.py`
-

Phase 2: Safety & Quality (Week 3-4)

2.1 Branch Protection Enforcement

Priority: P1 | Effort: 2-3 hours

```
iskra --protect-branches      # Enable protection
iskra --auto-branch feat/auto # Auto-create feature branch
```

Tasks:

- ☐ Check current branch before operations
- ☐ **Big warning** if on protected branch:
WARNING: You are on protected branch 'main'

Protected branches: main, master, production

Options:
 1. Continue anyway (requires typing 'yes')
 2. Create feature branch (iskra will create 'auto/2024-11-28')
 3. Cancel operation
- Choose [1/2/3]:
- ☐ Add to config:

```
protected_branches: [main, master, production]
strict_branch_protection: false # If true, block completely
auto_create_branch: true
branch_prefix: "auto/"
```
- ☐ Auto-create feature branch if requested
- ☐ Return branch info in JSON/API
- ☐ Add tests

Files to modify:

- src/auto_commit/core/git_operations.py
 - src/auto_commit/config.py
-

2.2 Pre-Commit Validation

Priority: P1 | Effort: 4-5 hours

```
iskra --validate      # Run all checks
iskra --no-validate   # Skip checks
```

Tasks:

☐ Create `src/auto_commit/validation/` module

☐ Implement validators:

```
class PreCommitValidator:
    def check_large_files(self, max_size_mb: int = 50) -> List[str]
    def scan_secrets(self) -> List[SecretMatch]
    def check_debug_code(self) -> List[DebugStatement]
    def check_todos(self) -> List[TodoComment]
    def check_conflicts(self) -> List[ConflictFile]
    def run_custom_command(self, cmd: str) -> CommandResult

    def validate_all(self) -> ValidationResult
```

☐ Secret patterns to detect:

- AWS keys: `AKIA[0-9A-Z]{16}`
- API keys: `api[_-]?key.*['"] [0-9a-zA-Z]{32,}['"]`
- Tokens: `token.*['"] [0-9a-zA-Z]{32,}['"]`
- Private keys: `-----BEGIN.*PRIVATE KEY-----`

☐ Debug patterns:

- JavaScript: `console.log`, `debugger`
- Python: `print()`, `pdb.set_trace()`, `breakpoint()`
- Go: `fmt.Println`, `log.Println`

☐ Display validation results:

Pre-commit Validation Issues:

BLOCKING Issues (must fix):

- Large file detected: `data/large.csv` (125 MB)
- Secret found in `config.py:42` (API key pattern)
- Merge conflict in `src/app.py`

WARNINGS (can proceed):

- Debug statement in `utils.js:15` (`console.log`)
- TODO comment in `main.py:89`

Continue anyway? [y/N]:

☐ Configurable severity levels

☐ `.iskraignore` file for validation exclusions

☐ Return results in JSON

☐ Add to API

Files to create:

- `src/auto_commit/validation/__init__.py`

- `src/auto_commit/validation/validators.py`
 - `src/auto_commit/validation/patterns.py`
-

2.3 Operation History & Rollback

Priority: P2 | Effort: 4-5 hours

```
iskra history           # Show recent operations
iskra undo              # Rollback last operation
iskra undo --repo PATH  # Rollback specific repo
iskra undo --dry-run    # Preview what would be undone
```

Tasks:

- ☐ Create history log: `~/.config/iskra/history.json`

```
{
  "operations": [
    {
      "id": "uuid",
      "timestamp": "2024-11-28T14:30:00",
      "operation": "batch_commit",
      "repos": [
        {
          "path": "/path/to/repo",
          "before_commit": "abc123",
          "after_commit": "def456",
          "pushed": true,
          "branch": "main"
        }
      ]
    }
  ]
}
```

- ☐ Log every operation in `IskraManager`
- ☐ Implement undo logic:
 - Reset to previous commit: `git reset --hard <before_commit>`
 - If pushed: warn user, require `--force`
 - If pushed to protected: block completely
- ☐ `iskra history` command shows recent operations
- ☐ Limit history to last 100 operations
- ☐ Add to API
- ☐ Handle edge cases (detached HEAD, etc.)

Files to create:

- src/auto_commit/history/manager.py
-

2.4 Better Diff Handling

Priority: P2 | Effort: 3-4 hours

```
iskra diff                # Show diffs for all repos
iskra diff --repo PATH    # Single repo diff
iskra diff --json         # Structured diff data
iskra --show-diff         # Show before committing
```

Tasks:

- ☐ Create `iskra diff` subcommand
- ☐ Rich colored diff output using `rich.syntax`
- ☐ Paginate long diffs (prompt to continue)
- ☐ Summary header: +125 -43 lines across 8 files
- ☐ File tree of changes:

```
Changes in my-repo:
  src/
    main.py      (+15 -5)
    utils.py     (+45 -0)
  tests/
    test_main.py (+30 -10)
```

- ☐ JSON output format:

```
{
  "repo": "/path/to/repo",
  "files": [
    {
      "path": "src/main.py",
      "status": "modified",
      "additions": 15,
      "deletions": 5,
      "diff": "..."
    }
  ],
  "summary": {
    "files_changed": 3,
    "insertions": 90,
    "deletions": 15
  }
}
```

```
}  
}
```

- ☐ Interactive prompt after diff: Commit? [Y/n/e(dit)]
- ☐ Add to API

Files to create:

- src/auto_commit/diff/ module
-

Phase 3: Advanced Features (Month 2)

3.1 Enhanced AI Commit Messages

Priority: P2 | Effort: 6-8 hours

Tasks:

- ☐ Parse issue numbers from branch names:
 - feat/JIRA-123-login → feat(auth): add login [JIRA-123]
 - fix/gh-42-bug → fix: resolve bug (#42)
- ☐ Breaking change detection:
 - Large deletions (>100 lines)
 - API signature changes
 - Config format changes
 - Add BREAKING CHANGE: footer
- ☐ Generate commit body:

feat(api): add user authentication

 - Implement JWT token generation
 - Add login and logout endpoints
 - Update user model with password hashing
 - Add authentication middleware

Breaking Change: API now requires auth header
- ☐ Multi-file context:
 - “Updated 3 API endpoints and added corresponding tests”
 - “Refactored database layer across 5 modules”
- ☐ Style learning:
 - Analyze last 20 commits in repo
 - Extract patterns (tense, format, emoji usage)
 - Match repo’s style

- ☐ Update Go binary `ai_commit` with enhanced prompts
- ☐ Add configuration options:

```
commit_style:
  include_body: true
  detect_breaking_changes: true
  parse_issue_numbers: true
  learn_from_history: true
```

Files to modify:

- `python/gocli/cmd/auto_commit/main.go`
 - `python/gocli/internal/llm/ollama.go`
-

3.2 Multi-Provider AI Support

Priority: P2 | Effort: 5-6 hours

```
iskra --ai-provider claude
iskra --ai-provider openai
iskra --ai-provider ollama # Default
```

Tasks:

- ☐ Create provider interface:


```
class AIProvider(ABC):
    @abstractmethod
    def generate_commit_message(self, diff: str, context: dict) -> str
```
- ☐ Implement providers:
 - `OllamaProvider` (existing)
 - `ClaudeProvider` (Anthropic SDK)
 - `OpenAIProvider` (OpenAI SDK)
 - `FallbackProvider` (tries multiple)
- ☐ Fallback chain:


```
ai_providers:
  - ollama
  - claude
  - openai
  - template # Generic template as last resort
```
- ☐ API key management:
 - Store in config: `~/.config/iskra/secrets.yaml` (gitignored)
 - Environment variables: `ANTHROPIC_API_KEY`, `OPENAI_API_KEY`
 - Secure storage (keyring library)

- ☐ Cost tracking for paid APIs:

```
{
  "usage": {
    "claude": { "requests": 150, "tokens": 45000, "cost": 0.23 },
    "openai": { "requests": 50, "tokens": 20000, "cost": 0.15 }
  }
}
```

- ☐ Add to config and API

Files to create:

- src/auto_commit/ai/providers/ module
- src/auto_commit/ai/providers/base.py
- src/auto_commit/ai/providers/ollama.py
- src/auto_commit/ai/providers/claude.py
- src/auto_commit/ai/providers/openai.py

Dependencies to add:

- anthropic>=0.7.0
 - openai>=1.0.0
-

3.3 Pull-Repos Update Mode

Priority: P2 | Effort: 3-4 hours

```
pull-repos --update           # Pull existing repos only
pull-repos --sync             # Clone new + pull existing
pull-repos --update --json    # For Zvezda
```

Tasks:

- ☐ Detect existing repos in base directory
- ☐ For existing repos: run `git pull` instead of clone
- ☐ Handle merge conflicts:
 - Detect conflicts
 - Skip repo with warning
 - Log to results
- ☐ Show differentiated results:

```
Clone Results:
Cloned 5 new repositories
Updated 20 existing repositories
2 conflicts (skipped)
1 failed
```

- ☐ Add to JSON output
- ☐ Configurable pull strategy:
 - --pull-rebase: Use `git pull --rebase`
 - --pull-merge: Use `git pull --merge`
 - --pull-ff-only: Use `git pull --ff-only`

Files to modify:

- `src/auto_commit/pull_repos.py`
 - `src/auto_commit/github/clone.py`
-

3.4 Configuration Profiles

Priority: P2 | Effort: 3-4 hours

```
iskra --profile work           # Use work profile
iskra --profile personal      # Use personal profile
iskra profile list            # List profiles
iskra profile set work        # Set default profile
iskra profile create PROJECT  # Create new profile
```

Tasks:

- ☐ Profile directory structure:


```
~/.config/iskra/
  config.yaml      # Global config
  profiles/
    work.yaml
    personal.yaml
    client-project.yaml
  repos.json       # Default profile repos
  repos-work.json  # Work profile repos
```
- ☐ Profile config schema:


```
name: work
base_dir: ~/Work
exclude_patterns: [personal-*, temp-*]
only_patterns: [client-*, internal-*]
ai_provider: claude
auto_push: true
require_confirmation: false
```
- ☐ Profile management commands:
 - `iskra profile list`
 - `iskra profile create NAME`

- `iskra profile delete NAME`
- `iskra profile set NAME` (set default)
- `iskra profile show [NAME]`

- ☐ Add to config manager
- ☐ Add to API
- ☐ Zvezda can list and switch profiles

Files to modify:

- `src/auto_commit/config.py`
 - `src/auto_commit/init.py` (add profile commands)
-

3.5 Batch Command Execution

Priority: P3 | Effort: 3-4 hours

```
iskra exec "npm update"
iskra exec --only "backend-*" "go mod tidy"
iskra exec --has-changes "npm test"
iskra exec --parallel "git fetch"
iskra exec --json "git status --short"
```

Tasks:

- ☐ Add `exec` subcommand
- ☐ Execute command in each repo directory
- ☐ Capture stdout/stderr per repo
- ☐ Display options:
 - Live streaming (default)
 - Summary at end
 - JSON output
- ☐ Execution modes:
 - Sequential (default)
 - Parallel (`--parallel`, with `--max-workers N`)
- ☐ Error handling:
 - `--fail-fast`: Stop on first error
 - `--continue`: Continue on errors (default)
- ☐ Results format:

Execution Results:

```

my-repo (0.5s)
npm install
added 5 packages

web-app (2.3s)
npm install
Error: Cannot find module 'react'

```

Summary: 12/15 successful

☐ JSON output:

```

{
  "command": "npm update",
  "results": [
    {
      "repo": "/path/to/repo",
      "success": true,
      "exit_code": 0,
      "stdout": "...",
      "stderr": "",
      "duration": 0.5
    }
  ]
}

```

☐ Add to API

Files to create:

- src/auto_commit/exec/ module

3.6 Conflict Resolution Helpers

Priority: P3 | Effort: 2-3 hours

```

iskra resolve --list           # List conflicts
iskra resolve --ours FILE     # Resolve with ours
iskra resolve --theirs FILE    # Resolve with theirs
iskra resolve --auto-ours     # Auto-resolve all with ours

```

Tasks:

- ☐ Detect conflict markers: <<<<<<, =====, >>>>>>
- ☐ List conflicted files
- ☐ Helper commands:
 - --ours: git checkout --ours <file>

- `--theirs: git checkout --theirs <file>`
- `--manual`: Open in `$EDITOR`

- ☐ Auto-resolution strategies (with confirmation)
- ☐ Show diff of conflicts before resolving
- ☐ Add to API for Zvezda integration

Files to create:

- `src/auto_commit/conflicts/` module
-

Phase 4: Integration & Polish (Month 3)

4.1 Report Generation

Priority: P3 | Effort: 4-5 hours

```
iskra report                                # Last 7 days
iskra report --period 30                    # Last 30 days
iskra report --json                          # For Zvezda
iskra report --format html                  # HTML report
```

Tasks:

- ☐ Analyze operation history
- ☐ Generate statistics:
 - Total commits made
 - Commits per repo
 - Commits per day (graph)
 - Most active repos
 - Lines changed (+/-)
 - Average commit message length
 - AI provider usage
- ☐ Output formats:
 - Terminal (Rich tables/graphs)
 - JSON (for Zvezda)
 - HTML (standalone file)
 - CSV (for spreadsheets)
- ☐ Visualizations:
 - ASCII bar chart of activity
 - Heatmap of commit frequency
 - Repo activity breakdown

Files to create:

- `src/auto_commit/reports/` module
-

4.2 Git Worktree Support

Priority: P3 | Effort: 3-4 hours

Tasks:

- ☐ Detect worktrees: `git worktree list`
- ☐ List all worktrees in status
- ☐ Process each worktree as independent unit
- ☐ Avoid duplicate operations (main + worktree)
- ☐ Show worktree info in JSON output
- ☐ Add to API

Files to modify:

- `src/auto_commit/core/git_operations.py`
-

4.3 Multi-Remote Support

Priority: P3 | Effort: 3-4 hours

```
iskra --push-to origin,backup # Push to multiple remotes
iskra status --show-remotes   # Show all remotes
```

Tasks:

- ☐ Detect all remotes: `git remote -v`
- ☐ Push to multiple remotes in sequence
- ☐ Show remote status for each remote (ahead/behind)
- ☐ Fork sync: fetch from upstream
- ☐ Configure default remotes in `.iskra.yaml`
- ☐ Add to JSON output

Files to modify:

- `src/auto_commit/core/git_operations.py`
-

4.4 Notifications

Priority: P3 | Effort: 3-4 hours

```
iskra --notify # Enable notifications
```

Tasks:

- ☐ Desktop notifications:

- Linux: `notify-send`
- macOS: `osascript`- Windows: `win10toast`

☐ Webhook integrations:

- Slack: POST to webhook URL
- Discord: POST with embed
- Custom: Generic webhook

☐ Email notifications (optional):

- SMTP configuration
- HTML email template
- Summary report

☐ Configuration:

```

notifications:
  enabled: true
  desktop: true
webhooks:
  slack: "https://hooks.slack.com/..."
  discord: "https://discord.com/api/webhooks/..."
email:
  smtp_server: "smtp.gmail.com"
  from: "iskra@example.com"
  to: ["user@example.com"]

```

☐ Notification content:

- Operation summary
- Success/failure counts
- Error details
- Duration

Files to create:

- `src/auto_commit/notifications/` module

4.5 Performance Optimization

Priority: P3 | Effort: Ongoing

Tasks:

- ☐ Profile slow operations
- ☐ Parallel processing where safe:
 - Status checks
 - Pre-commit validation

- Read-only operations
 - ☐ Caching:
 - Status results (with TTL)
 - Remote comparisons
 - File change detection
 - ☐ Progress indicators for long operations
 - ☐ Timeout handling for hung operations
 - ☐ Resource limits (max concurrent operations)
-

ZVEZDA TODO (TUI Dashboard)

Phase 1: Foundation (Week 1-2)

1.1 Project Setup

Priority: P0 | Effort: 2-3 hours

Tasks:

- ☐ Create repository: github.com/NoamFav/Zvezda
- ☐ Initialize Go module: `go mod init zvezda`
- ☐ Project structure:

```
zvezda/
├── cmd/
│   └── zvezda/
│       └── main.go
├── internal/
│   ├── ui/           # Bubble Tea UI
│   ├── iskra/        # Iskra integration
│   ├── state/        # Application state
│   └── config/        # Zvezda config
├── pkg/
│   └── models/       # Shared types
├── go.mod
├── go.sum
└── README.md
```

- ☐ Install dependencies:

```
// go.mod
require (
    github.com/charmbracelet/bubbletea v0.25.0
```

```

github.com/charmbracelet/lipgloss v0.9.1
github.com/charmbracelet/bubbles v0.17.1
github.com/muesli/termenv v0.15.2
)

```

- ☐ Create basic README
- ☐ Setup .gitignore
- ☐ Create LICENSE

1.2 Iskra Integration Layer

Priority: P0 | Effort: 4-6 hours | Blockers: Iskra 1.1, 1.2

Interface with Iskra (both CLI and Python API).

Tasks:

- ☐ Create internal/iskra/client.go:

```

type IskraClient struct {
    pythonPath string
    configPath string
}

// CLI execution methods
func (c *IskraClient) GetStatus() ([]RepoStatus, error)
func (c *IskraClient) ProcessRepo(path string, opts ProcessOptions) error
func (c *IskraClient) ProcessAll(opts ProcessOptions) error
func (c *IskraClient) GetTrackedRepos() ([]RepoInfo, error)

// Execute iskra commands and parse JSON output
func (c *IskraClient) executeJSON(args []string) (interface{}, error)

```

- ☐ Alternative: Python C-API integration (more complex but better performance):

```

// Using go-python or similar
import "github.com/go-python/gpython/py"

type IskraPythonClient struct {
    manager *py.Object // IskraManager instance
}

```

- ☐ Decision: Start with CLI execution, optimize later
- ☐ Parse Iskra's JSON output
- ☐ Handle Iskra errors gracefully

- ☐ Add retries for transient failures
- ☐ Timeout handling

Files to create:

- `internal/iskra/client.go`
 - `internal/iskra/types.go` (mirror Iskra's types)
 - `internal/iskra/parser.go` (JSON parsing)
-

1.3 Basic Bubble Tea Application

Priority: P0 | Effort: 4-5 hours

Get a minimal TUI running.

Tasks:

- ☐ Create main Bubble Tea model:

```
type Model struct {
    iskra      *iskra.Client
    repos      []RepoInfo
    cursor     int
    selected   map[int]struct{}
    loading    bool
    err        error
    currentView ViewType
}

type ViewType int
const (
    ViewDashboard ViewType = iota
    ViewRepoList
    ViewRepoDetail
    ViewLogs
    ViewSettings
)
```

- ☐ Implement Bubble Tea interface:

```
func (m Model) Init() tea.Cmd
func (m Model) Update(msg tea.Msg) (tea.Model, tea.Cmd)
func (m Model) View() string
```

- ☐ Basic keyboard navigation:

- ↑/k: Up
- ↓/j: Down
- Enter: Select

- **Space:** Toggle
- **q/Ctrl+C:** Quit
- **?:** Help
- **Tab:** Switch view

☐ Simple views:

- Welcome screen
- Loading spinner
- Error display

☐ Color scheme using Lipgloss

Files to create:

- internal/ui/model.go
- internal/ui/update.go
- internal/ui/view.go
- internal/ui/keys.go
- cmd/zvezda/main.go

Phase 2: Core Views (Week 3-4)

2.1 Dashboard View

Priority: P1 | **Effort:** 6-8 hours | **Blockers:** 1.2, 1.3

Main overview screen.

Tasks:

☐ Create dashboard layout:

Zvezda Dashboard

Repository Summary

Total Repositories: 50

Clean: 32

Changes: 12

Behind: 6

Ahead: 8

Conflicts: 0

Recent Activity (Last 7 Days)

Mon 12 commits

Tue 18 commits

Wed 6 commits

Thu 14 commits

Fri 22 commits

Quick Actions
[p] Process All [s] Status [r] Refresh [v] View List

Press '?' for help 'q' to quit

- ☐ Repository summary cards
- ☐ Activity graph (ASCII bar chart)
- ☐ Top active repositories list
- ☐ Quick actions bar
- ☐ Auto-refresh every 30s (configurable)
- ☐ Smooth transitions between updates

Files to create:

- internal/ui/views/dashboard.go
- internal/ui/components/summary.go
- internal/ui/components/graph.go

2.2 Repository List View

Priority: P1 | Effort: 6-8 hours | Blockers: 2.1

Scrollable, filterable list of all repos.

Tasks:

- ☐ Create list view:

Repository List (50 repos)

Filters: [all] clean changes behind ahead

Sort by: [name] status activity

Search: my-project_

my-project	main	Clean	2h	
web-app	dev	5 changes	1d	
api-service	feat/*	12 changes	3h	← Selected
mobile-app	main	Behind ↓3	5h	
backend-api	main	Ahead ↑2	12h	
utils-lib	main	Clean	2d	
...				

[Enter] Details [Space] Select [a] Select All
[p] Process Selected [/] Search [f] Filter

- ☐ Virtual scrolling for large lists
- ☐ Multi-select with checkboxes
- ☐ Fuzzy search (using `github.com/sahilm/fuzzy`)
- ☐ Filter by state (clean, changes, behind, ahead)
- ☐ Sort options (name, status, last activity)
- ☐ Color coding:
 - Green: Clean
 - Yellow: Has changes
 - Red: Conflicts
 - Blue: Behind remote
 - Magenta: Ahead remote

☐ Batch operations on selected repos

☐ Vim-like keybindings (j/k, gg/G)

Files to create:

- `internal/ui/views/list.go`
- `internal/ui/components/repoitem.go`
- `internal/ui/components/filters.go`

2.3 Repository Detail View

Priority: P1 | Effort: 5-6 hours | Blockers: 2.2

Detailed view of a single repository.

Tasks:

- ☐ Create detail view:

Repository: my-project

/home/user/Projects/my-project

Branch: main (protected)

Remote: `github.com/user/my-project`

Status:

```
Uncommitted Changes: 5 files
• src/main.py      (modified)
• tests/test_main.py (modified)
• README.md        (modified)
• config.yaml      (added)
• old.py           (deleted)

Behind Remote: 3 commits
Ahead Remote: 2 commits

Last Commit (2h ago):
feat(api): add user authentication
by John Doe <john@example.com>
abc123def

Actions:
[p] Pull  [c] Commit  [P] Push  [d] Diff  [l] Log

[Esc] Back to list
```

- ☐ Show all repository details
- ☐ File change list with icons
- ☐ Last commit info with diff stats
- ☐ Remote status (ahead/behind)
- ☐ Action buttons for common operations
- ☐ Show validation warnings (large files, secrets, etc.)
- ☐ Worktree information
- ☐ Multiple remotes
- ☐ Real-time updates

Files to create:

- internal/ui/views/detail.go
 - internal/ui/components/statuscard.go
 - internal/ui/components/commitinfo.go
-

2.4 Diff Viewer

Priority: P2 | Effort: 5-6 hours | Blockers: 2.3

View diffs with syntax highlighting.

Tasks:

- ☐ Create diff viewer:

Diff: src/main.py

```
@@ -15,7 +15,12 @@ def main():

    def process_data(data):
-        return data.upper()
+        # Process and validate data
+        if not data:
+            raise ValueError("Data cannot be empty")
+        return data.strip().upper()

    def save_result(result):
        with open('output.txt', 'w') as f:
            f.write(result)

+5 lines -1 lines

[n] Next file  [p] Previous  [Esc] Back
```

- ☐ Syntax highlighting (using github.com/alecthomas/chroma)
- ☐ Scrollable diff view
- ☐ Navigate between files
- ☐ Show diff stats per file
- ☐ Copy diff to clipboard
- ☐ Export diff to file

Files to create:

- internal/ui/views/diff.go
- internal/ui/components/diffview.go

Phase 3: Advanced Features (Week 5-6)

3.1 Interactive Operations

Priority: P1 | Effort: 6-8 hours | Blockers: 2.2

Run Iskra operations from TUI with progress.

Tasks:

☐ Create operation flow:

1. Select repos (multi-select in list view)
2. Choose operation (commit, push, pull, etc.)
3. Confirm operation
4. Show progress with live updates
5. Display results

☐ Operation modal:

Process Repositories

Selected: 5 repositories

Operations:

- ☒ Pull latest changes
- ☒ Commit changes (AI-generated messages)
- ☒ Push to remote
- ☐ Create pull requests

Options:

- ☐ Dry run (preview only)
- ☒ Stop on error
- ☐ Skip confirmation

[Enter] Confirm [Esc] Cancel

☐ Progress view:

Processing...

Progress: 3/5 repositories

[] 60%

Current: web-app

Pulled latest changes

Committed: "feat(ui): add dark mode"

→ Pushing to remote...

Completed:

my-project (2.3s)

api-service (1.8s)

web-app (3.1s) ← current

Pending:

- mobile-app

- backend-api

[Ctrl+C] Cancel

- ☐ Use Iskra's event hooks for real-time updates
- ☐ Show live output from Iskra
- ☐ Handle errors gracefully
- ☐ Allow cancellation
- ☐ Summary screen after completion

Files to create:

- internal/ui/views/operation.go
 - internal/ui/views/progress.go
 - internal/ui/components/progressbar.go
-

3.2 Watch Mode

Priority: P1 | Effort: 5-6 hours | Blockers: 3.1

Monitor repositories for changes and auto-commit.

Tasks:

- ☐ File system watcher (using github.com/fsnotify/fsnotify)
- ☐ Watch all tracked repositories
- ☐ Debounce file changes (wait 30s after last change)
- ☐ Auto-commit when changes detected
- ☐ Watch mode UI:

Watch Mode (Active)

Watching 50 repositories

Recent Activity:

```

14:32:15  my-project      3 files changed
           → Waiting 30s...

14:28:42  web-app          5 files changed
           Committed: "feat: add feature"

14:15:33  api-service      2 files changed

```

Committed: "fix: resolve bug"

Statistics:

Auto-commits today: 12

Files monitored: 1,234

Uptime: 2h 15m

[s] Stop watching [p] Pause [c] Configure

- ☐ Configurable per-repo (respect `.iskra.yaml`)
- ☐ Exclude patterns (node_modules, etc.)
- ☐ Notification on auto-commit
- ☐ Pause/resume watching
- ☐ Watch statistics
- ☐ Background daemon mode

Files to create:

- `internal/watcher/watcher.go`
 - `internal/ui/views/watch.go`
-

3.3 Log Viewer

Priority: P2 | Effort: 4-5 hours

View operation logs and history.

Tasks:

- ☐ Create log viewer:

Operation Log

Filter: [all] errors warnings info

Search: commit_

```
14:32:15 [INFO] Processing 5 repositories
14:32:16 [INFO] my-project: Committed abc123
14:32:17 [WARN] web-app: Large file detected
14:32:18 [ERROR] api-service: Merge conflict
14:32:19 [INFO] Completed: 4/5 successful
...
```

[f] Filter [/] Search [c] Clear [e] Export

- ☐ Read Iskra's log files
- ☐ Real-time log streaming
- ☐ Filter by level (info, warn, error)
- ☐ Search logs
- ☐ Color coding by level
- ☐ Export logs
- ☐ Tail mode (follow newest)

Files to create:

- internal/ui/views/logs.go
 - internal/logs/reader.go
-

3.4 Settings/Configuration View

Priority: P2 | Effort: 4-5 hours

Manage Zvezda and Iskra configuration.

Tasks:

- ☐ Create settings view:

Settings

Zvezda Settings

Theme:	[Dark] Light Auto
Refresh interval:	[30] seconds
Key bindings:	[Vim] Emacs Default
Notifications:	[x] Enabled

Iskra Settings

AI Provider:	[Ollama] Claude OpenAI
Auto-push:	[x] Enabled
Require confirm:	[x] Enabled
Protected branches:	main, master, production

Profiles

Active profile: [Default] Work Personal
Base directory: ~/Projects

[s] Save [r] Reset [Esc] Cancel

- ☐ Edit Zvezda config
- ☐ Edit Iskra config (through API)
- ☐ Profile management
- ☐ Theme customization
- ☐ Keybinding customization
- ☐ Validate settings
- ☐ Apply changes without restart

Files to create:

- internal/ui/views/settings.go
 - internal/config/zvezda.go
-

Phase 4: Polish & Distribution (Week 7-8)

4.1 Help System

Priority: P2 | Effort: 2-3 hours

Comprehensive help and documentation.

Tasks:

- ☐ Help modal:

Keyboard Shortcuts

Navigation

↑/k	Move up
↓/j	Move down
g/Home	Go to top
G/End	Go to bottom
Ctrl+u	Page up
Ctrl+d	Page down

Actions	
Enter	Select/Open
Space	Toggle selection
a	Select all
A	Deselect all

Views	
d	Dashboard
l	Repository list
s	Settings
L	Logs
w	Watch mode

General	
?	Toggle help
r	Refresh
q/Ctrl+C	Quit

[Esc] Close help

- ☐ Context-sensitive help
- ☐ Command palette (fuzzy search for actions)
- ☐ Tutorial on first run
- ☐ Tooltips for complex features

Files to create:

- internal/ui/views/help.go
 - internal/ui/components/tooltip.go
-

4.2 Themes & Customization

Priority: P3 | Effort: 3-4 hours

Tasks:

- ☐ Theme system using Lipgloss
- ☐ Built-in themes:
 - Dark (default)
 - Light
 - Gruvbox
 - Nord
 - Solarized

- Dracula
 - Monokai
- ☐ Custom theme support (JSON/YAML)
- ☐ Color picker in settings
- ☐ Live theme preview
- ☐ Save theme preferences

Files to create:

- `internal/ui/themes/themes.go`
 - `internal/ui/themes/defaults.go`
-

4.3 Testing

Priority: P2 | Effort: Ongoing

Tasks:

- ☐ Unit tests for all modules
 - ☐ Integration tests with Iskra
 - ☐ TUI snapshot tests (if possible)
 - ☐ Performance tests
 - ☐ Mock Iskra client for testing
 - ☐ CI/CD pipeline
-

4.4 Documentation

Priority: P1 | Effort: 4-6 hours

Tasks:

- ☐ Comprehensive README:
 - Installation
 - Screenshots/GIFs
 - Features overview
 - Keyboard shortcuts
 - Configuration
- ☐ User guide
- ☐ Developer documentation
- ☐ Architecture diagrams
- ☐ Contributing guide

- ☐ Changelog
-

4.5 Distribution

Priority: P1 | Effort: 3-4 hours

Tasks:

- ☐ Build system:
 - `make build` *# Build binary*
 - `make install` *# Install locally*
 - `make release` *# Create release binaries*
 - ☐ Cross-platform builds (Linux, macOS, Windows)
 - ☐ Installation script
 - ☐ Homebrew formula (macOS)
 - ☐ AUR package (Arch Linux)
 - ☐ Snap/AppImage (Linux)
 - ☐ Release automation (GitHub Actions)
-

Phase 5: Advanced Features (Month 3+)

5.1 Analytics Dashboard

Priority: P3 | Effort: 6-8 hours

Tasks:

- ☐ Detailed commit analytics
 - ☐ Repository activity heatmap
 - ☐ Contributor statistics (if multi-user)
 - ☐ Trend analysis
 - ☐ Custom reports
 - ☐ Export analytics
-

5.2 Plugin System

Priority: P3 | Effort: 8-10 hours

Tasks:

- ☐ Plugin architecture
- ☐ Load external plugins

- ☐ Plugin API
 - ☐ Example plugins:
 - Jira integration
 - Slack notifications
 - Custom validators
-

5.3 Team Features

Priority: P3 | Effort: TBD

Tasks:

- ☐ Shared configuration
 - ☐ Team activity view
 - ☐ Collaboration features
 - ☐ Access control
-

Recommended Execution Order

Month 1: Foundation

Week 1: Iskra Foundation

1. JSON output mode (Iskra)
2. Python library interface (Iskra)
3. Event hooks system (Iskra)

Week 2: Zvezda Foundation 4. Zvezda project setup 5. Iskra integration layer (Zvezda) 6. Basic Bubble Tea app (Zvezda)

Week 3: Core Views 7. Status query command (Iskra) 8. Smart filters (Iskra) 9. Dashboard view (Zvezda)

Week 4: Polish 10. Repository list view (Zvezda) 11. Repository detail view (Zvezda) 12. Branch protection (Iskra)

Month 2: Features

Week 5-6:

- Pre-commit validation (Iskra)
- Interactive operations (Zvezda)
- Diff viewer (Zvezda)
- Watch mode (Zvezda)

Week 7-8:

- Enhanced AI (Iskra)
- Multi-provider AI (Iskra)
- Log viewer (Zvezda)
- Settings view (Zvezda)